CS 360
Programming Languages
Day 21

Review

- stacker
- funstacker
- Readers/Expanders
- Syntax/Semantics

Project 4

- Make a turtle graphics language based on a real language call Logo

- You'll have a turtle (a list) which the language modifies and uses to draw

- Most of the syntax is very simple, except…
  - **REPEAT … END**

A New Language: **bf**

- **helloworld.bf**

```
++++++[>+++++++++++++<-]>.
>++++++++++[>++++++++++<-]>+.
+++++++..+++.>++++[>++++++++++++<-]>.
<+++[>----<-]>.<<<<<+++[>+++++<-]>.
>>.+++.------.--------.>>+.
```

bf is Turing Complete

- Create an array of 30,000 bytes and a pointer to the first byte, then:
- **>:** increase pointer to next byte
- **<:** decrease pointer to previous byte
- **+:** increase current byte by 1
- **-:** decrease current byte by 1
- **.:** write the current byte to output
- **,:** read a byte from input
- **[...]:** repeat the instructions until the current byte is 0

Nested Structures in Languages: the Problem

- How do you “tell” Racket that there is a nested syntax structure?
- Shouldn’t you be able to nest nested structures inside nested structures?
- It’s complicated! You can try figuring it out on Project 4 “manually”!

Nested Structures in Languages: the Solution

- **Grammar!**

- Grammar of a Zip Code:

```
zip-code : digit digit digit digit digit
digit    : "0" | "1" | "2" | "3" | "4" | "5" | "6"
         | "7" | "8" | "9"
```

- You try: make a grammar of an email address (make simplifying assumptions if you like!)

Can’t you just do that with Regular Expressions?

- Yes and no! What can a grammar do that regular expressions can’t?

Recursive Grammars

This grammar generates any addition expression:

```
expression : (expression + expression) | integer
```

Let’s generate:

```
(4 + 6)
((4 + 6)+ ((2 + 1) + 3))
```

More Grammars

- Stacker/Funstacker’s Grammar

- Racket’s Grammar

- bf’s grammar

Stacker

```
stacker-program : "\n"* instruction ("\n"+ instruction)*
instruction     : integer | func
integer         : ["-"] digit+
digit           : "0" | "1" | "2" | "3" | "4"
                | "5" | "6" | "7" | "8" | "9"
func            : "+" | "*"
```

Use this grammar to produce a stacker program:

```
5
-2
*
```

Funstacker

- Use the following grammar to produce some math-expressions:

```
m-expr  : m-list | integer
m-list  : "(" func (" "+ m-expr)* ")"
integer : ["-"] digit+
digit   : "0" | "1" | "2" | "3" | "4"
        | "5" | "6" | "7" | "8" | "9"
func    : "+" | "*"
```

Try `(* 2 (+ 3 4) 5)`

Try `(+ (* -2 3 4) 1)`

bf

- Produce the following programs with the following grammar:

```
>>>,++.
```

```
+[>+]>.
```

```
bf-program : (bf-op | bf-loop)*
bf-op      : ">" | "<" | "+" | "-" | "." | ","
bf-loop    : "[" bf-program "]"
```

Racket Grammar

Exercise:

- Make a grammar that produces a valid assignment in Python. Be as complete as possible!

- Examples:

```
my_var = my_val # valid!
1my_var = my_val # invalid!
my_var = my_val + # invalid!
```